UNIVERSIDAD DE PALERMO

Microservicios y APIs Escalables · 2026

SPRINT 10

Eureka Server · Config Server · Spring Boot Actuator

Completando ms-dueno + Infraestructura de Soporte

Fase 2 — Microservicios (Sprints 9-14)
Clases 9 (parte 2) y 10 | Duración estimada: 2 clases
Entregable: infraestructura levantada (Eureka + Config) + ms-dueno completado y registrado
Stack nuevo: Spring Cloud 2025.x · Spring Cloud Netflix Eureka · Spring Cloud Config
Prerequisito: estructura Maven multi-módulo del Sprint 9 creada y validada

1. Objetivo del Sprint

En el Sprint 9 construimos el esqueleto hexagonal de ms-dueno: dominio y application. En este sprint completamos los adaptadores (web + persistence), levantamos ms-dueno en el puerto 8081, y agregamos la infraestructura de soporte que necesitan los microservicios para encontrarse entre sí y compartir configuración.

Al finalizar este sprint van a poder:

- Completar los adaptadores web y persistence de ms-dueno con arquitectura hexagonal.
- Levantar Eureka Server y registrar ms-dueno en él.
- Configurar Config Server con un repositorio Git de configuración.
- Exponer y consultar endpoints de Actuator para monitoreo.
- Replicar la estructura hexagonal en los otros tres microservicios.

2. Arquitectura del sistema al finalizar el sprint

Al terminar este sprint, el ecosistema de microservicios quedará así:

Componente	Puerto	Tecnología	Rol
eureka-server	8761	Spring Cloud Netflix	Service Discovery — registro y localización de servicios
config-server	8888	Spring Cloud Config	Configuración centralizada para todos los microservicios
ms-dueno	8081	Spring Boot 4.1.0	Bounded Context: gestión de dueños
ms-mascota	8082	Spring Boot 4.1.0	Bounded Context: gestión de mascotas
ms-veterinario	8083	Spring Boot 4.1.0	Bounded Context: gestión de veterinarios
ms-turno	8084	Spring Boot 4.1.0	Bounded Context: gestión de turnos

Flujo de arranque obligatorio:

1. config-server (debe estar UP antes que cualquier microservicio)
2. eureka-server (debe estar UP antes que los microservicios)
3. ms-dueno, ms-mascota, ms-veterinario, ms-turno (en cualquier orden)

Si un microservicio arranca antes que Config Server, falla al obtener su configuración.
Si arranca antes que Eureka, no puede registrarse (pero reintentará automáticamente).

3. Completar ms-dueno — Adaptadores (Sprint 9 continuación)

PASO 1 Adaptador de salida — DuenoJpaRepository

-> Crear DuenoJpaRepository en adapter/out/persistence/DuenoJpaRepository.java
-> Extender JpaRepository<Dueno, Long> (del paquete org.springframework.data.jpa.repository)
-> Implementar la interface DuenoRepositoryPort (port de salida del dominio)
-> Anotar con @Repository
-> Delegar cada método al this (JpaRepository ya los provee):
findAll() -> this.findAll()
findById(Long id) -> this.findById(id) [ya devuelve Optional<Dueno>]
save(Dueno dueno) -> this.save(dueno)
deleteById(Long id) -> this.deleteById(id)
existsByEmail(String email) -> declarar el método: Spring Data lo genera solo
-> Verificar que DuenoJpaRepository NO importa nada de domain.port.in ni de application

PASO 2 Adaptador de entrada — DTOs

-> Crear DuenoRequest en adapter/in/web/dto/DuenoRequest.java:
Campos: nombre (@NotBlank), apellido (@NotBlank), email (@Email @NotBlank), telefono (@NotBlank)
Anotar la clase con @Data, @NoArgsConstructor, @AllArgsConstructor
-> Crear DuenoResponse en adapter/in/web/dto/DuenoResponse.java:
Campos: id (Long), nombre, apellido, email, telefono
Anotar igual con @Data, @NoArgsConstructor, @AllArgsConstructor
-> Crear DuenoMapper en adapter/in/web/dto/DuenoMapper.java:
Método estático toEntity(DuenoRequest req) -> Dueno
Método estático toResponse(Dueno dueno) -> DuenoResponse
(sin MapStruct por ahora — mapeo manual para claridad pedagógica)

PASO 3 Adaptador de entrada — DuenoController

-> Crear DuenoController en adapter/in/web/DuenoController.java
-> Anotar con @RestController, @RequestMapping("/api/duenos"), @RequiredArgsConstructor
-> Inyectar DuenoService (la interface del dominio, NO DuenoServiceImpl)
-> Implementar los endpoints:
GET /api/duenos -> 200 OK + List<DuenoResponse>

GET /api/duenos/{id}   -> 200 OK + DuenoResponse
POST /api/duenos   -> 201 Created + DuenoResponse (@Valid en @RequestBody)
PUT /api/duenos/{id}   -> 200 OK + DuenoResponse
DELETE /api/duenos/{id} -> 204 No Content
-> Usar DuenoMapper para convertir entre entidades y DTOs
-> El controller NO maneja excepciones directamente (las atrapa el @ControllerAdvice)

PASO 4 GlobalExceptionHandler (ControllerAdvice)

-> Crear GlobalExceptionHandler en adapter/in/web/GlobalExceptionHandler.java
-> Anotar con @RestControllerAdvice
-> Crear ErrorResponse record con campos: timestamp, status, error, mensaje, path
-> Manejar:
DuenoNotFoundException -> 404 Not Found
DuplicatedEmailException -> 409 Conflict
MethodArgumentNotValidException -> 400 Bad Request (errores de @Valid)
Exception -> 500 Internal Server Error (fallback)
-> Cada handler construye un ErrorResponse con LocalDateTime.now() como timestamp

PASO 5 application.yml y primera ejecución de ms-dueno

-> Crear src/main/resources/application.yml en ms-dueno:

```
spring:
  application:
    name: ms-dueno
  datasource:
    url: jdbc:mysql://localhost:3306/dueno_db
    username: root
    password: (tu password)
    driver-class-name: com.mysql.cj.jdbc.Driver
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true
server:
  port: 8081
```

-> IMPORTANTE: NO agregar spring.jpa.database-platform (Hibernate 7 lo detecta solo)
-> Crear la BD en MySQL Workbench: CREATE DATABASE dueno_db;
-> Crear MsDuenoApplication.java en com.patitas.dueno con @SpringBootApplication
-> Levantar: mvn spring-boot:run -pl ms-dueno
-> Verificar Swagger en http://localhost:8081/swagger-ui/index.html
-> Probar un POST /api/duenos con datos válidos y verificar respuesta 201

4. Replicar la estructura en los otros microservicios

Una vez que ms-dueno funciona, replicar la misma estructura hexagonal en los otros tres módulos. El procedimiento es idéntico:

Para ms-mascota (puerto 8082, BD: mascota_db):
Entidad Mascota: id, nombre, especie, raza, fechaNacimiento, duenoId (Long — referencia por ID, no @ManyToOne)
Port in: MascotaService con findAll, findById, findByDuenoId, save, update, deleteById
Excepción de dominio: MascotaNotFoundException

Para ms-veterinario (puerto 8083, BD: veterinario_db):
Entidad Veterinario: id, nombre, apellido, matricula, especialidad, email
Port in: VeterinarioService con findAll, findById, findByEspecialidad, save, update, deleteById

Para ms-turno (puerto 8084, BD: turno_db):
Entidad Turno: id, fechaHora, mascotaId, veterinarioId, motivo, estado (PENDIENTE/CONFIRMADO/CANCELADO)
Port in: TurnoService con findAll, findById, findByVeterinarioId, findByMascotaId, save, cancelar

IMPORTANTE: los IDs de otras entidades se guardan como Long (duenoId, mascotaId, veterinarioId)
NO usar @ManyToOne entre microservicios — cada uno tiene su propia BD.

Por qué NO usamos @ManyToOne entre microservicios:

En el monolito, Mascota tenia @ManyToOne Dueno porque compartian la misma BD.
En microservicios, ms-mascota y ms-dueno tienen BDs distintas.
No hay foreign key cruzada posible — cada servicio es dueno de sus datos.
La referencia se hace por ID (Long duenoId) y la validacion se hace via HTTP (Sprint 13).

5. Eureka Server — Service Discovery

5.1 ¿Para qué sirve Eureka?

Cuando ms-turno necesita hablar con ms-veterinario, ¿cómo sabe dónde está? No puede hardcodear "localhost:8083" porque en producción las IPs cambian. Eureka resuelve esto: cada microservicio se registra con su nombre y Eureka actúa como directorio telefónico.

Sin Eureka: ms-turno llama a http://localhost:8083/api/veterinarios/1
Hardcodeado. Roto en cualquier entorno que no sea local.

Con Eureka: ms-turno llama a http://ms-veterinario/api/veterinarios/1
Eureka traduce "ms-veterinario" a la IP real. Funciona en cualquier entorno.

PASO 6 Crear el módulo eureka-server

-> En el pom.xml PADRE agregar el módulo: <module>eureka-server</module>
-> Crear la carpeta eureka-server/ con su pom.xml referenciando al padre
-> En el pom.xml de eureka-server agregar las dependencias:
spring-cloud-starter-netflix-eureka-server
spring-boot-starter-actuator
-> Agregar en el pom.xml PADRE la dependencia de Spring Cloud (BOM):
En <dependencyManagement> agregar:
<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-dependencies</artifactId>
<version>2025.0.0</version>
<type>pom</type>
<scope>import</scope>
</dependency>
-> Crear EurekaServerApplication.java con @SpringBootApplication y @EnableEurekaServer
-> Crear application.yml de eureka-server:
server.port: 8761
spring.application.name: eureka-server
eureka.client.register-with-eureka: false
eureka.client.fetch-registry: false
eureka.server.enable-self-preservation: false
-> Levantar: mvn spring-boot:run -pl eureka-server
-> Verificar dashboard en http://localhost:8761

PASO 7 Registrar ms-dueno en Eureka

-> En el pom.xml de ms-dueno agregar:
spring-cloud-starter-netflix-eureka-client
-> En application.yml de ms-dueno agregar:
eureka:
client:
service-url:
defaultZone: http://localhost:8761/eureka/
instance:
prefer-ip-address: true
-> Levantar ms-dueno (con eureka-server ya corriendo)
-> Verificar en http://localhost:8761 que aparece MS-DUENO en la lista de instancias
-> Repetir para ms-mascota, ms-veterinario y ms-turno cuando estén listos

6. Config Server — Configuración centralizada

6.1 ¿Para qué sirve Config Server?

Hoy cada microservicio tiene su propio application.yml con la URL de la BD, el puerto de Eureka, etc. Si cambia la URL de Eureka, hay que modificar 4 archivos y redeployar 4 servicios. Config Server centraliza toda la configuración en un repositorio Git.

Flujo de Config Server:

1. Config Server se conecta a un repositorio Git al arrancar
2. Los microservicios le preguntan al Config Server su configuración al arrancar
3. Config Server les responde con el contenido del archivo correspondiente del repo Git
4. Si cambias la config en Git y haces refresh, los microservicios la obtienen sin restart

PASO 8 Crear el módulo config-server

-> En el pom.xml PADRE agregar el módulo: <module>config-server</module>
-> Crear la carpeta config-server/ con su pom.xml referenciando al padre
-> Dependencia: spring-cloud-config-server
-> Crear ConfigServerApplication.java con @SpringBootApplication y @EnableConfigServer
-> Crear application.yml de config-server:
server.port: 8888
spring.application.name: config-server
spring.cloud.config.server.git.uri: https://github.com/TU_USER/patitas-config
spring.cloud.config.server.git.default-label: main
spring.cloud.config.server.git.clone-on-start: true

PASO 9 Crear el repositorio de configuración Git

-> Crear un nuevo repositorio público en GitHub llamado patitas-config
-> En ese repositorio crear los archivos de configuración:
ms-dueno.yml (configuración de ms-dueno)
ms-mascota.yml
ms-veterinario.yml
ms-turno.yml
application.yml (config compartida por todos)
-> En application.yml del repo de config poner la URL de Eureka:
eureka.client.service-url.defaultZone: http://localhost:8761/eureka/
-> En ms-dueno.yml del repo de config poner la config de la BD:
spring.datasource.url: jdbc:mysql://localhost:3306/dueno_db
spring.datasource.username: root
spring.datasource.password: (password)
-> Levantar config-server: mvn spring-boot:run -pl config-server
-> Verificar: GET http://localhost:8888/ms-dueno/default -> debe devolver el YAML del repo

PASO 10 Conectar ms-dueno al Config Server

-> En ms-dueno reemplazar el application.yml con un bootstrap.yml (o usar application.yml con import):

```
spring:
  application:
    name: ms-dueno
  config:
    import: "optional:configserver:http://localhost:8888"
server:
  port: 8081
```

-> El resto de la configuración (datasource, eureka, jpa) viene del Config Server

-> Agregar dependencia en pom.xml de ms-dueno: spring-cloud-starter-config

-> Levantar en orden: config-server -> eureka-server -> ms-dueno

-> Verificar que ms-dueno arranca y obtiene su config correctamente

-> Verificar que sigue apareciendo en el dashboard de Eureka

7. Spring Boot Actuator — Monitoreo

7.1 ¿Qué es Actuator?

Actuator expone endpoints HTTP que muestran el estado interno de la aplicación. Es la primera herramienta de observabilidad que activamos.

Endpoint	Qué muestra
/actuator/health	Estado UP/DOWN del servicio + BD + dependencias
/actuator/info	Información del servicio (nombre, versión, descripción)
/actuator/metrics	Métricas JVM, HTTP, memoria, CPU
/actuator/env	Variables de entorno y configuración activa
/actuator/beans	Todos los beans de Spring registrados
/actuator/mappings	Todos los endpoints HTTP registrados

PASO 11 Configurar Actuator en ms-dueno

-> La dependencia spring-boot-starter-actuator ya está en el pom.xml del padre

-> En el application.yml del repo de config (ms-dueno.yml), agregar:

```
management:
  endpoints:
    web:
      exposure:
        include: health,info,metrics,env,beans,mappings
  endpoint:
    health:
      show-details: always
  info:
```

```
    env:
      enabled: true
  info:
    app:
      name: ms-dueno
      description: Microservicio de gestión de dueños - Patitas Felices
      version: 1.0.0
-> Reiniciar ms-dueno y verificar:
  GET http://localhost:8081/actuator/health  ->  {"status":"UP"}
  GET http://localhost:8081/actuator/info    ->  datos del servicio
  GET http://localhost:8081/actuator/metrics ->  lista de métricas disponibles
```

8. Criterios de aceptación del Sprint

eureka-server corre en puerto 8761 y muestra el dashboard
config-server corre en puerto 8888 y responde GET /ms-dueno/default con el YAML correcto
ms-dueno arranca obteniendo su config desde Config Server
ms-dueno aparece registrado en el dashboard de Eureka
GET http://localhost:8081/actuator/health devuelve {"status":"UP"}
GET http://localhost:8081/actuator/info devuelve datos del servicio
Todos los endpoints CRUD de /api/duenos siguen funcionando (Swagger)
La Dependency Rule se mantiene: DuenoServiceImpl no importa nada de adapter.*
Los otros tres microservicios tienen la estructura hexagonal creada (aunque sea vacía)

9. Entregable del Sprint 10

- Repositorio patitas-felices con los módulos: eureka-server, config-server, ms-dueno, ms-mascota, ms-veterinario, ms-turno.
- Repositorio patitas-config en GitHub con los archivos de configuración.
- ms-dueno completamente funcional con arquitectura hexagonal + registrado en Eureka.
- Screenshot o video corto del dashboard de Eureka con ms-dueno registrado.
- README actualizado con instrucciones de orden de arranque.

10. Vista previa — Sprint 11

En el Sprint 11 vamos a:

- Incorporar el API Gateway con Spring Cloud Gateway (WebFlux).
- Enrutar todas las requests externas a través del gateway en puerto 8080.
- Implementar Circuit Breaker con Resilience4J para tolerancia a fallos.
- Ver cómo el gateway consulta Eureka para resolver los nombres de los servicios.